

HW4 – Traveling Salesperson Problem

Branch and Bound

Student ID: 112000112 Name:

Design and Analysis of Algorithms

1 Pseudocode of the Branch-and-Bound Algorithm

The algorithm performs a depth-first search over the space of partial Hamiltonian paths starting at city 1. At every node it maintains a lower bound on the cost of completing the tour; if that bound is no better than the current best tour, the subtree is pruned.

Lower bound

For every city i , let $\text{minOut}[i] = \min_{j \neq i} C[i][j]$. Any complete tour must use exactly one outgoing edge from each city, so the cost of any tour is at least $\sum_i \text{minOut}[i]$. When the search has fixed a partial path $v_0, v_1, \dots, v_{d-1}$ with $\text{current} = v_{d-1}$, the outgoing edges of $v_0, \dots, v_{d-2}$ are already chosen (their cost is already in cost), but the outgoing edges of current and of every unvisited city are not yet determined. Hence the remaining cost is at least

$$\text{remMin} = \text{minOut}[\text{current}] + \sum_{u \notin \text{visited}} \text{minOut}[u].$$

remMin is maintained incrementally: when the algorithm extends the path with an edge from current to next , current 's outgoing edge becomes fixed, so $\text{remMin}' = \text{remMin} - \text{minOut}[\text{current}]$.

Pseudocode

```
Algorithm TSP-BranchAndBound(C, n)
Input :   n x n cost matrix C
Output:   A shortest Hamiltonian cycle starting and ending at city 1,
          and its total cost.

/* ----- 1. Pre-computation ----- */
for i = 1 to n
    minOut[i] <- min_{j != i} C[i][j]
totalMin <- sum_{i=1..n} minOut[i]

/* ----- 2. Initial upper bound (nearest neighbour) ----- */
visited[1] <- true, visited[2..n] <- false
cur <- 1, cost <- 0, path <- (1)
for k = 2 to n
    pick the unvisited city j with smallest C[cur][j]
    append j to path, visited[j] <- true
    cost <- cost + C[cur][j]; cur <- j
bestCost <- cost + C[cur][1]
bestPath <- path

/* ----- 3. Depth-first branch and bound ----- */
```

```

visited[1] <- true,  curPath <- (1)
DFS(1, 1, 0, totalMin)

output bestPath, bestCost

Procedure  DFS(current, depth, cost, remMin)

/* base case: every city has been placed */
if depth = n then
    total <- cost + C[current][1]
    if total < bestCost then
        bestCost <- total;  bestPath <- curPath
    return

/* current's outgoing edge is about to be chosen, so it leaves remMin */
newRemMin <- remMin - minOut[current]

/* expand the children sorted by edge cost (best-first) */
cand <- empty list
for next = 1 to n
    if not visited[next] then
        append (C[current][next], next) to cand
sort cand in ascending order of the first coordinate

for each (edge, next) in cand
    newCost <- cost + edge
    /* PRUNE: best achievable from this branch is newCost + newRemMin */
    if newCost + newRemMin >= bestCost then
        break          /* later candidates are worse, so cut them all */
    visited[next] <- true
    append next to curPath
    DFS(next, depth + 1, newCost, newRemMin)
    remove last element of curPath
    visited[next] <- false

```

Correctness and Complexity

- **Correctness.** $cost + remMin$ is a valid lower bound on the cost of any completion of the current partial tour, because each remaining outgoing edge contributes at least its row-minimum. Pruning a subtree whose lower bound is at least $bestCost$ therefore cannot discard the optimum. An initial upper bound is obtained from a feasible nearest-neighbour tour.
- **Worst-case complexity.** $O(n!)$ – the same as exhaustive search. In practice the bound and the nearest-neighbour seed prune the vast majority of branches, giving the running times reported below.

2 Running-time Comparison

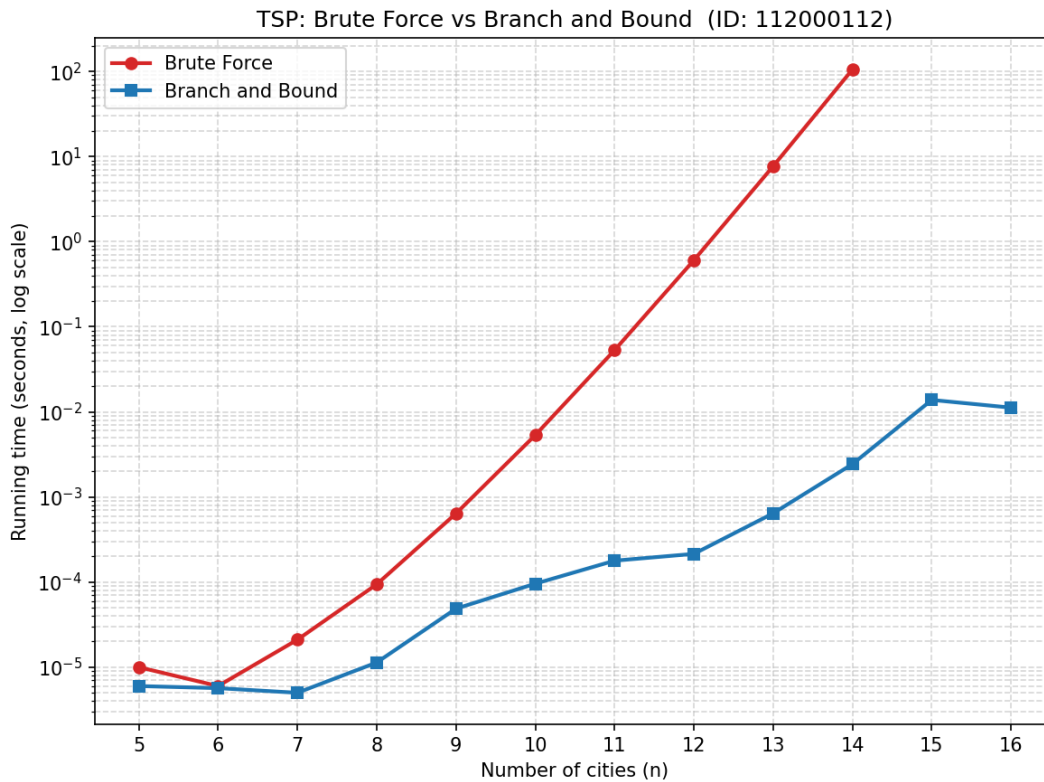
Random instances were generated with edge costs drawn uniformly from $[1, 9999]$ (diagonal 10000) for each $n \in \{5, 6, \dots, 16\}$. For every n the branch-and-bound program was run on **3** random instances and the mean execution time was recorded; the brute-force program was run on **1** instance per n . All timings were measured by `clock()` inside the program (search only –

I/O excluded). Brute force is $O(n!)$ and becomes infeasible from $n = 15$; data points beyond $n = 14$ are omitted from its curve.

Raw timings (seconds)

n	Brute Force	Branch & Bound (avg of 3)
5	0.000010	0.000006
6	0.000006	0.000006
7	0.000021	0.000005
8	0.000094	0.000011
9	0.000639	0.000049
10	0.005324	0.000096
11	0.053115	0.000178
12	0.605313	0.000214
13	7.706000	0.000643
14	105.621755	0.002436
15	— (omitted)	0.013810
16	— (omitted)	0.011212

Plot



The brute-force curve grows by roughly a factor of n per step, exactly as predicted by $(n - 1)!$. The branch-and-bound curve grows much more slowly; its small dip from $n = 15$ to $n = 16$ is a typical sampling effect when pruning becomes especially effective on a particular random instance.

Hardware / software. CPU: Apple M-series (local), grader CPU: Intel i7-10700 / CentOS Linux 8. Compiler: `g++ -std=c++11 -O2`. Plot produced with `matplotlib` on Python 3.